Scaler GPU workloads with Argo Workflows & Karpenter

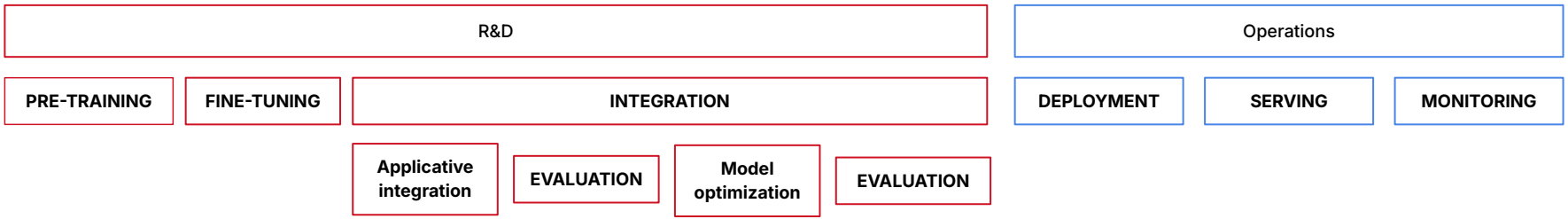
Amine Saboni , @aminesaboni.bsky.social
Pruna AI



Agenda

- ML models release cycle
- Karpenter: new generation autoscaler
- Orchestrate with Argo Workflows
- GPU-type aware orchestration patterns

ML model release cycle



ML model release cycle

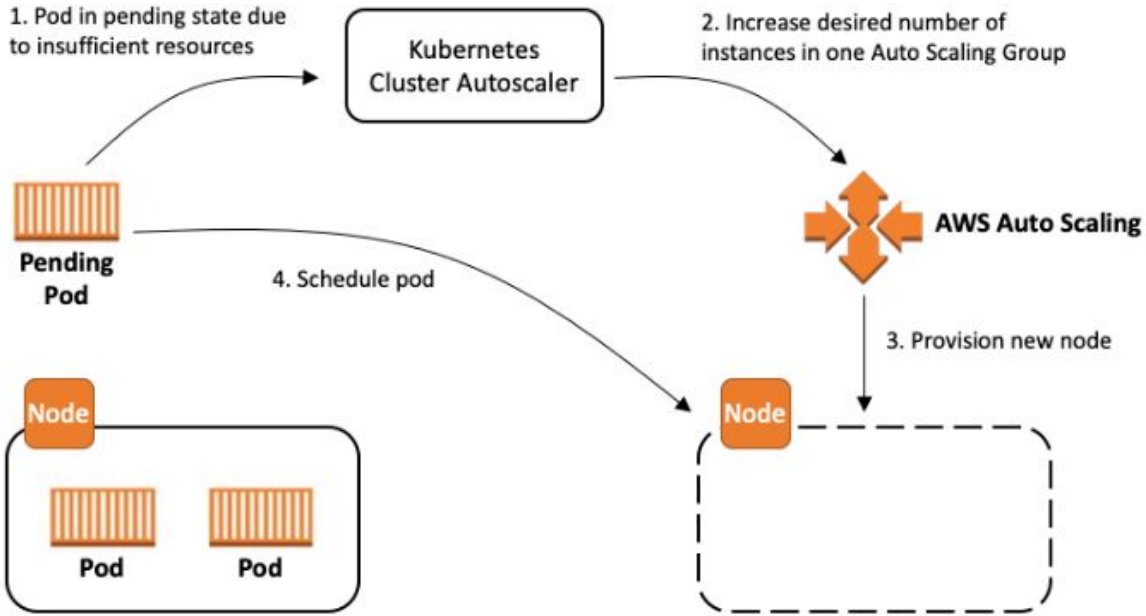


R&D						Operations		
PRE-TRAINING	**FINE-TUNING**	**INTEGRATION**				**DEPLOYMENT**	**SERVING**	**MONITORING**
External H100 / B200 Massive clusters	On cloud A100 / H100 1 MIG, multiple experiments	**Applicative integration** On cloud Standard GPU 1 instance	**EVALUATION** On cloud Specific GPU Single instance, multiple experiments	**Model optimization** On cloud Specific GPU 1 instance	**EVALUATION** On cloud Specific GPU Single instance		On cloud Specific GPU Multiple instances, single model	

Agenda

- ML models release cycle
- **Karpenter: new generation autoscaler**
- Orchestrate with Argo Workflows
- GPU-type aware orchestration patterns

Cluster autoscaler, the legacy solution

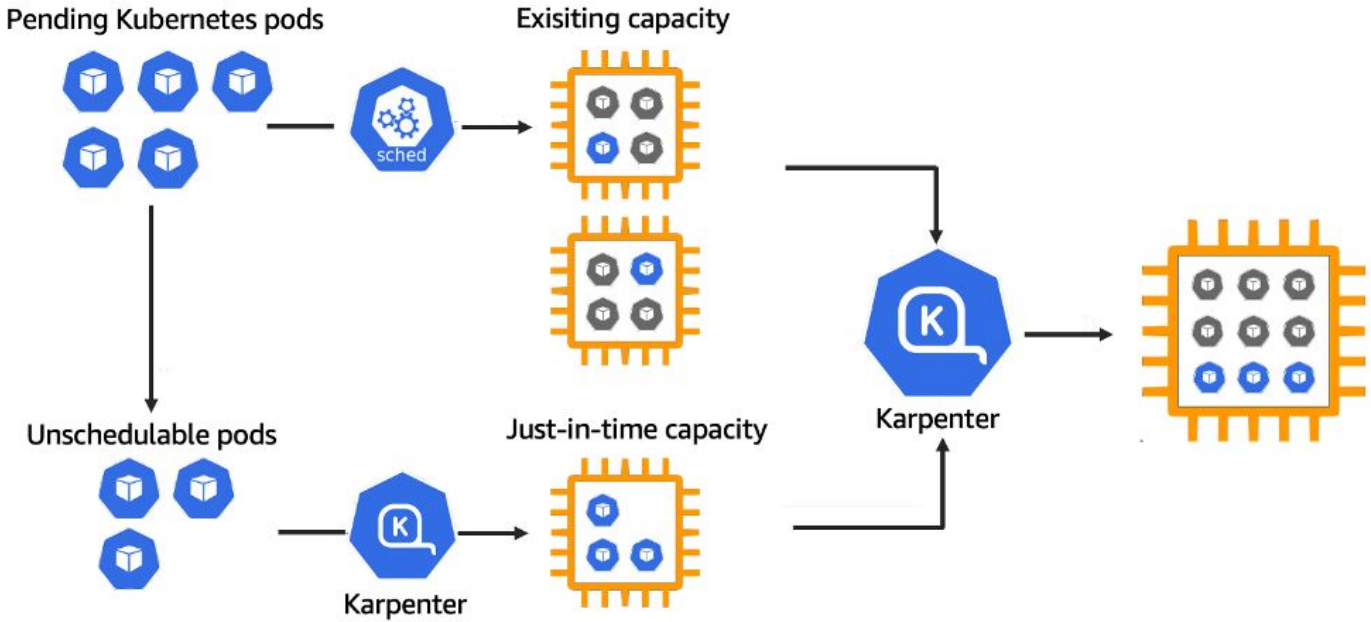


No scale to 0

Heterogenous fleets hard to manage

Using AWS NodeGroups

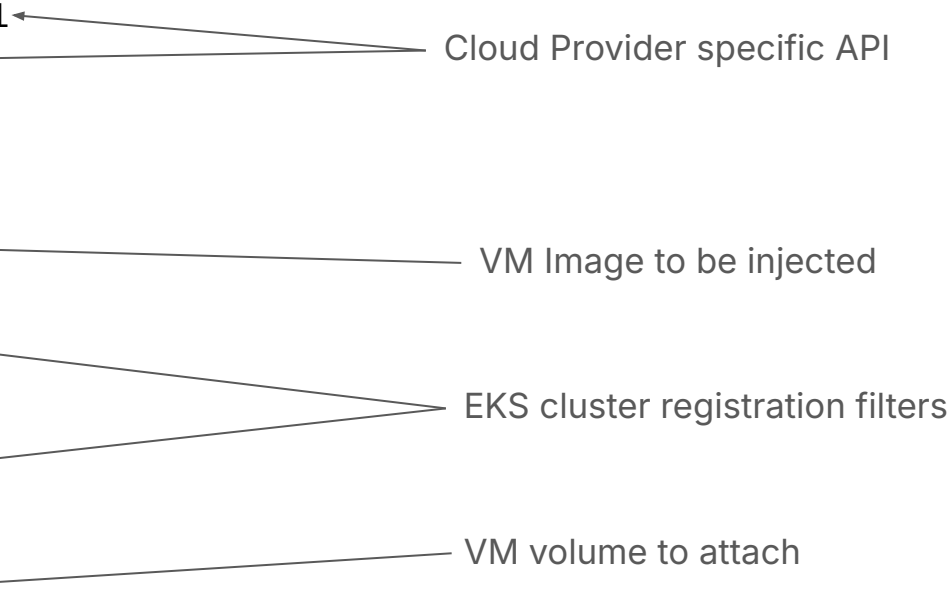
Karpenter, the new generation autoscaler from AWS



NodeClass

```
apiVersion: karpenter.k8s.aws/v1
kind: EC2NodeClass
metadata:
 name: gpu-pool
spec:
 amiFamily: AL2023

 subnetSelectorTerms:
   - tags:
       karpenter.sh/discovery:
 securityGroupSelectorTerms:
   - tags:
       karpenter.sh/discovery:
blockDeviceMappings:
```



NodePool

```
apiVersion: karpenter.sh/v1
kind: NodePool
spec:
 nodeClassRef:
  group: karpenter.k8s.aws
  kind: EC2NodeClass
  name: gpu-pool
requirements:
 - key: node.kubernetes.io/instance-type
   operator: In
   values:
   - g6e.xlarge
   - g4dn.2xlarge
 - key: "karpenter.sh/capacity-type"
   operator: In
   values: ["on-demand"]
```

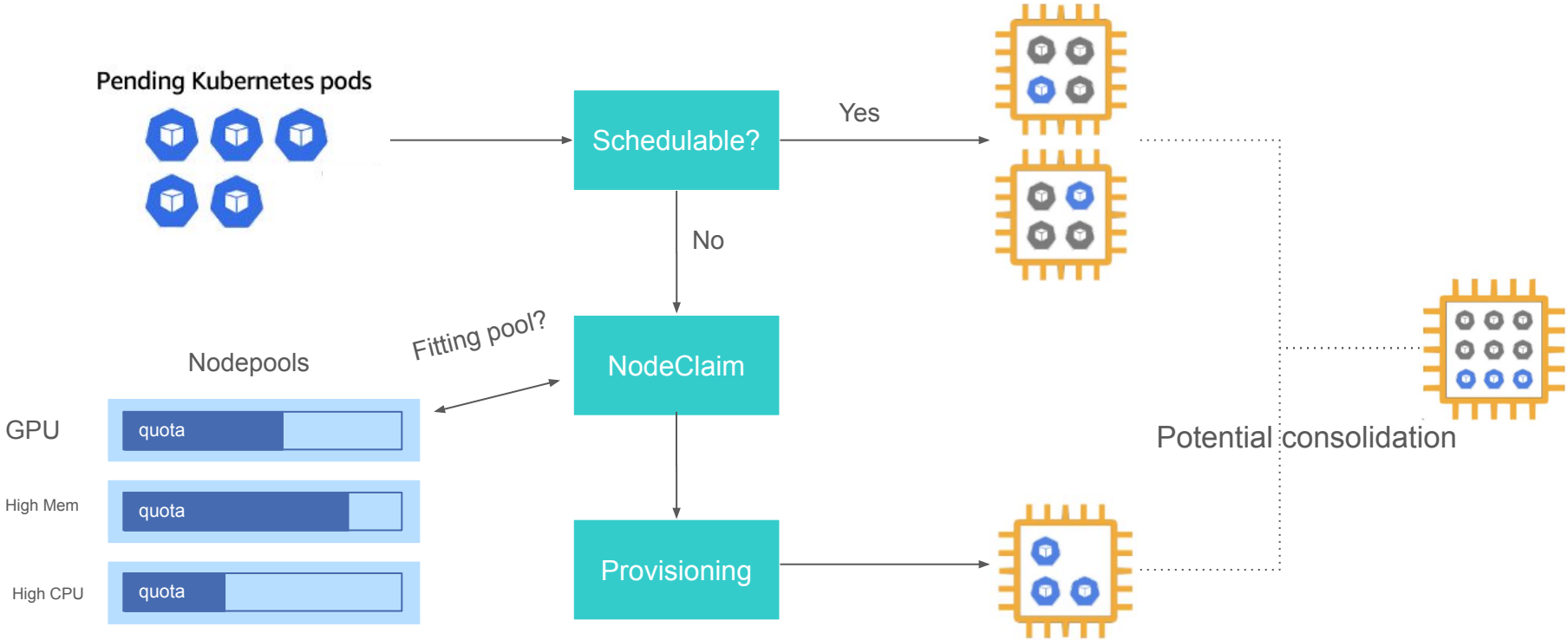
Standard API

References the nodeclass

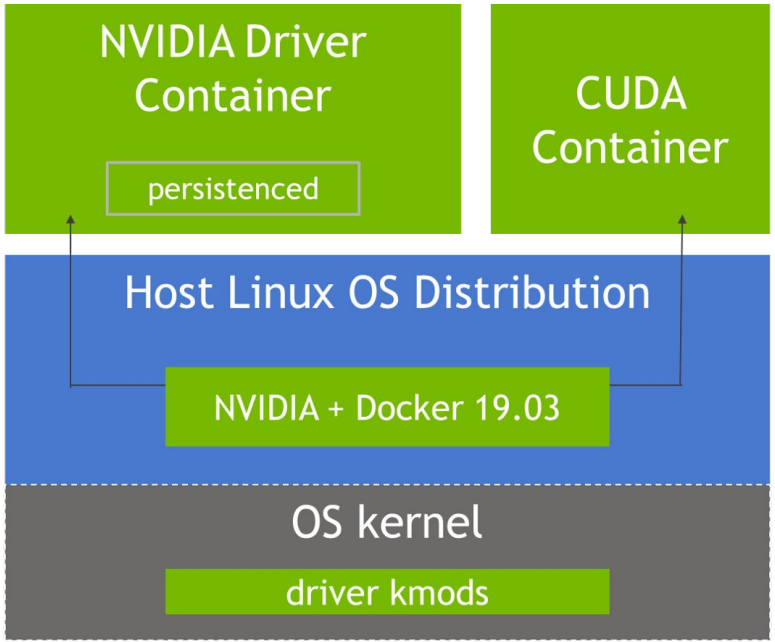
Types that can be used to spawn new instances

Additional filters for selection

Karpenter resolution loop



What about GPUs ?



Driver NVIDIA is linked to OS

Compatible image served by the cloud provider

To be schedule pod will specify which taint it does need, letting Karpenter select a compatible flavour

Agenda

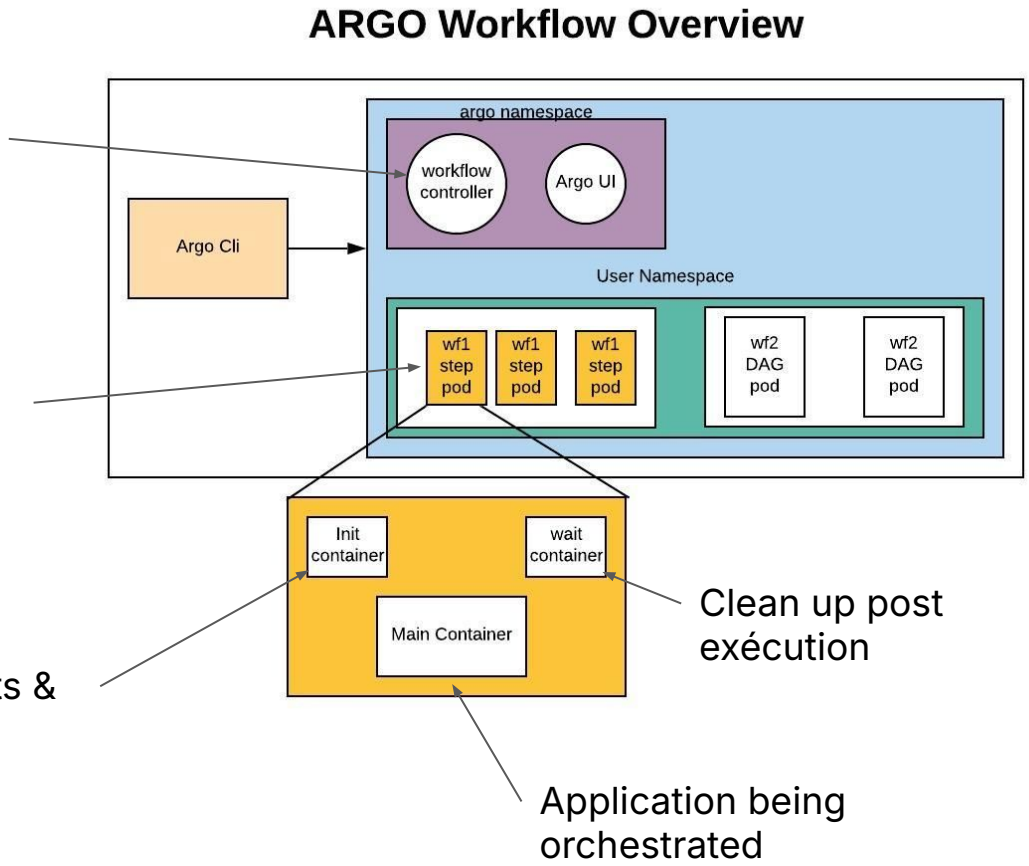
- ML models release cycle
- Karpenter: new generation autoscaler
- **Orchestrate with Argo Workflows**
- GPU-type aware orchestration patterns

An orchestrator made for developers

- Via CRDs, Argo Workflows creates & monitor Kube resources to guarantee proper execution of the flow

- Workflows, written by developers or data scientists, are directly orchestrated by the Kube engine : 1 step = 1 pod

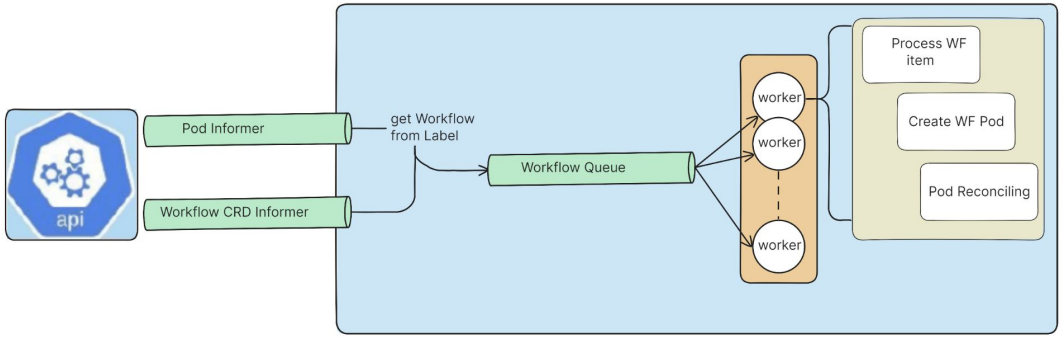
- Workflow are Kube resources that can be managed by ArgoCD



Workflow controller

Native integration with API Server :

- queuing of workflows / steps
- worker express its hardware needs
- after execution, Karpenter consolidates state of nodes requested by the workflow



Workflows templates

```yaml
apiVersion: argoproj.io/v1alpha1
kind: CronWorkflow
metadata:
 name: daily-hf-scan
spec:
 schedule: "0 0 * * *"
 concurrencyPolicy: Allow
 workflowSpec:
   serviceAccountName: argo-workflows-server
   entrypoint: main

 templates:
  - name: main
    steps:
      - - name: scan-hf
          template: scan-hf

      - - name: fan-out-quantization
          template: fan-out-quantization
          arguments:
            parameters:
              - name: sequence
                value: "{{steps.scan-hf.outputs.parameters.quantization_sequence}}"
```

Kube managed resource

Template defines interactions between steps, retry logic, fanout, artefacts / parameters passing between tasks, etc.

CLOUD NATIVE DAYS FRANCE 2026

Workflows templates

```
- name: scan-hf
  script:
    image: repository/image-name
    command: [ "/bin/bash", "-c" ]
    args:
      - |
        echo "Starting scanning"
        source .venv/bin/activate
        python hf_scan_job.py
    env:
      - name: HF_TOKEN
    nodeSelector:
      type: gpu-pool
    tolerations:
      - key: "nvidia.com/gpu"
        operator: "Equal"
        value: "true"
        effect: "NoSchedule"
    resources:
      requests:
        nvidia.com/gpu: 1
  outputs:
    parameters:
      - name: quantization_sequence
        valueFrom:
          path: /app/sequence.json
```

Also a resource managed by Kube

Karpenter managed NodePool

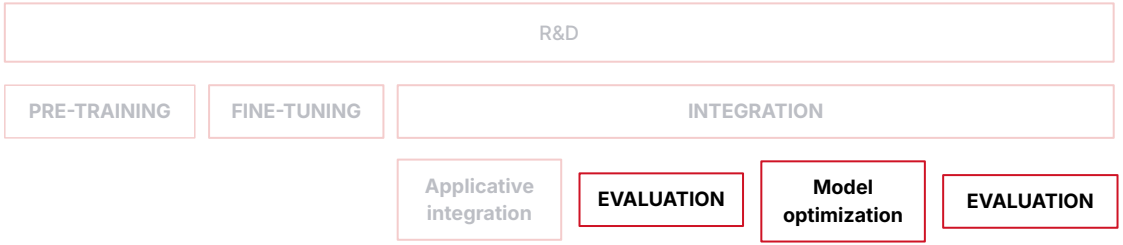
Provisioning constraints

Sequence express as a list of task for easy fanout

Agenda

- ML models release cycle
- Karpenter: new generation autoscaler
- Orchestrate with Argo Workflows
- **GPU-type aware orchestration patterns**

Heterogenous horizontal scaling



For optimisation or evaluation oriented tasks :

- Same script will be launched with multiple configurations
- hardware will be task-dependant (a small GPU will fit for benchmarking an already trained model, a bigger one might be necessary if a judge LLM must perform a more complex evaluation, etc.)

Heterogenous horizontal scaling

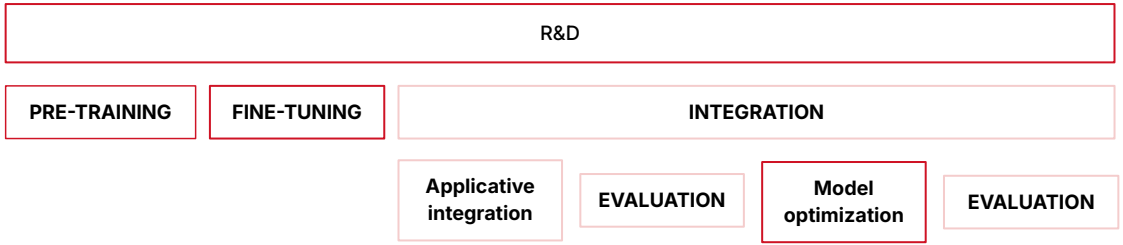
```
  outputs:
    parameters:
      - name: quantization_sequence
        valueFrom:
          path: /app/sequence.json

- name: fan-out-quantization
 inputs:
   parameters:
     - name: sequence
 dag:
   tasks:
     - name: do-smashing
       template: do-smashing
       withParam: "{{inputs.parameters.sequence}}"
       arguments:
         parameters:
             …
           - name: machine_type
             value: "{{item.machine_type}}"
       nodeSelector:
        type: "{{inputs.parameters.machine_type}}"
       resources:
        requests:
          nvidia.com/gpu: 1
```

The sequence has been computed with a gpu instance type specific to each item

This parameter is given as a nodeSelector type to schedule the pod on the appropriate node

Grid search benchmark




```yaml
# experiment.yaml
metadata:
  experiment_image: "my-experiment:latest"
  model:
  model_store: "/path/to/models"
  model_name: ["bert", "gpt2"]
evaluation:
  n_iterations: 1000
  metrics: ["accuracy", "latency"]
parameters_grid:
compression:
  method: ["quantize", "prune"]
  bits: [8, 16]
  learning_rate: [0.01, 0.001]
```

Set of parameters combination
All combinations must be tested, and results must be deuplicated in base

Grid search benchmark

```
tasks:
 - name: validate-configuration
   template: validate-configuration
   arguments:
     parameters:
       - name: parameters_grid
         value: "{{workflow.parameters.parameters_grid}}"

- name: run-experiment
 template: run-experiment
 arguments:
   parameters:
     - name: config_line
       value: "{{item}}"
 withParam: "{{tasks.validate-configuration.outputs.parameters.unique_combinations}}"
 depends: "validate-configuration"

- name: run-evaluate
 template: run-evaluate
 arguments:
  artifacts:
    - name: smashed-model
      from: "{{tasks.run-experiment.outputs.artifacts.smashed-model}}"
```

Deduplicate already tested set of parameters

Each configuration item is tested in parallel

Each config line will produce a different artefact, that is managed independently in the rest of pipeline

Thanks for your attention



Amine Saboni

amine.saboni@bsky.social

saboni.amine@gmail.com